# CONTINUATION-IN-PART: Pure Church Lambda Machine

## Church-Turing Meta-Machine: Architectural Exclusion of Turing-Domain Instructions as a Hardware Security Enforcement Mechanism

**Inventor**: Kenneth James Hamer-Hodges

**Parent Application**: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

**Classification**: Computer Architecture; Hardware Security; Capability-Based Computing; Lambda Calculus Processor; Vulnerability Elimination by Construction; Interactive Programming Model

## TITLE OF THE INVENTION

Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism in a Capability-Based Computer

## CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of the CTMM patent application filed February 12, 2026, which discloses the Golden Token capability architecture, domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions, the LAMBDA instruction for lightweight in-scope code application, and the atomic abstraction architecture. The present application extends that disclosure by demonstrating that the Turing domain can be entirely eliminated from the software instruction set without loss of computational completeness, and that this elimination constitutes a novel security enforcement mechanism.

## FIELD OF THE INVENTION

The present invention relates to a processor architecture that executes all software exclusively through Church's lambda calculus reduction operations, mediated by capability tokens (Golden Tokens), with Turing-domain instructions (arithmetic, branching, direct memory addressing) architecturally excluded from the instruction set available to software. A minimal hardware I/O mediator provides the sole interface between pure lambda software and physical devices.

## BACKGROUND

### The Vulnerability Problem

All known processor architectures provide Turing-domain instructions (ADD, SUB, MOV, LOAD/STORE to arbitrary addresses, branch to arbitrary targets) as part of the software-accessible instruction set.

This instruction set enables buffer overflow attacks (writing beyond allocated bounds via MOV/STORE), return-oriented programming (chaining existing code fragments via branch to gadget addresses), code injection (writing executable code to data regions then branching to it), and privilege escalation (manipulating kernel data structures via arbitrary memory access).

Software mitigations (ASLR, stack canaries, DEP/W^X, control flow integrity) reduce the probability of successful exploitation but cannot eliminate the underlying vulnerability: the instructions that enable the attacks remain available to software.

### The Capability Limitation

Capability-based architectures (Cambridge CAP, IBM System/38, CHERI, and the parent CTMM application) enforce access control through unforgeable tokens, requiring valid capabilities for every memory access. However, all prior capability architectures retain Turing-domain instructions for computation. While capabilities prevent unauthorized access to memory regions, they do not prevent misuse of legitimate access — a program with a valid write capability can still overflow a buffer within its authorized region, and Turing-domain branch instructions can still be chained for ROP attacks within the program's own code space.

### The Discovery

The parent CTMM application discloses domain purity enforcement: Golden Token permissions are separated into Turing domain (R, W, X) and Church domain (L, S, E), and a GT cannot have permissions from both domains simultaneously. This separation was designed for architectural cleanliness.

The present invention recognizes that domain purity can be extended to its logical conclusion: **an entire processor can operate with only Church-domain instructions available to software.** The inventor has demonstrated this through three complete proof implementations:

1. **HP-35 Scientific Calculator**: 179 instructions implementing the complete HP-35 (digit entry, four-function arithmetic, trigonometry via Taylor series, logarithms, exponentiation, square root via Newton-Y-combinator iteration, stack management, constant retrieval) — zero Turing-domain instructions.

2. **SlideRule Arithmetic Engine**: 98 instructions implementing 9 arithmetic operations (ADD, SUB, MUL, DIV, MOD, LOG, EXP, SQRT, POW) — zero Turing-domain instructions.

3. **Interactive Church Computer REPL (Read-Eval-Print Loop)**: A working Haskell implementation of the Pure Church Machine as an interactive programming environment, demonstrating that the architecture supports a complete programming model — not just fixed programs. The REPL executes arbitrary user-supplied Church-domain computations through the full 7-step capability-checked security pipeline (LOAD → TPERM → CALL → LOAD → TPERM → LAMBDA → RETURN), rejects all Turing-domain instructions with FAULT, and supports Ada Lovelace-style variable bindings for step-by-step named computation. A Bernoulli sum-of-squares program (following the style of Lovelace's Note G, 1843) demonstrates multi-step mathematical computation using only lambda calculus.

All implementations use exclusively six Church-domain opcodes: LOAD (L permission), SAVE (S permission), CALL (E permission), RETURN, LAMBDA (X permission), and TPERM (permission verification). Every computation is performed through Church-encoded lambda reductions: arithmetic via Church numerals, control flow via Church booleans, data structures via Church pairs, recursion via Y-combinator.

# DETAILED DESCRIPTION

## The Pure Church Instruction Set

The pure Church processor provides exactly six instructions to software:

| Instruction | Permission | Function |
|---|---|---|
| LOAD | L (Load) | Load a Golden Token from a C-List slot into a capability register |
| SAVE | S (Save) | Write data through a capability to a C-List slot |
| CALL | E (Enter) | Enter an abstraction scope — cross protection domain boundary |
| RETURN | — | Return from CALL or LAMBDA to caller's scope |
| LAMBDA | X (Execute) | Apply a Church function — lightweight in-scope code application |
| TPERM | — | Verify Golden Token permissions before use — fault on failure |

No other instructions exist in the software-accessible instruction set. Specifically, the following instruction classes are architecturally excluded:

- **Arithmetic**: ADD, SUB, MUL, DIV, MOD and all variants
- **Logic**: AND, OR, XOR, NOT, shift, rotate
- **Comparison**: CMP, CMN, TST, TEQ and all condition codes
- **Branching**: B, BL, BX, conditional branches, computed jumps
- **Direct memory access**: LDR, STR, LDM, STM to arbitrary addresses
- **Register transfer**: MOV between arbitrary registers

## How Computation Works Without Turing Instructions

All computation is performed through Church-encoded lambda calculus:

**Arithmetic**: Numbers are represented as Church numerals — the number N is the function that applies its first argument N times to its second argument. Addition applies f (m+n) times; multiplication composes applications; exponentiation applies multiplication repeatedly. These are not software libraries running on top of Turing instructions — they are the primitive computational mechanism of the machine.

**Control flow**: Church booleans (TRUE = λx.λy.x, FALSE = λx.λy.y) select between alternatives. The IF function simply applies the boolean to two branches. No branch instructions or condition codes exist.

**Data structures**: Church pairs (PAIR = λa.λb.λf. f a b) and selectors (FST, SND) provide structured data. Lists and trees are built from nested pairs.

**Recursion**: The Y-combinator (Y = λf. (λx. f(x x))(λx. f(x x))) enables recursive computation without loop instructions or goto. The SlideRule's SQRT uses Y-combinator-driven Newton iteration; LOG uses Y-combinator-driven iterative division.

**Method dispatch**: The method selector (DR1) is converted to a Church numeral by applying SUCC DR1 times to ZERO (= FALSE), then used to index into the abstraction's C-List. No jump table or branch

instruction is required.

### The I/O Mediator

A pure Church machine cannot directly interact with physical hardware because hardware registers are inherently stateful and side-effecting. The I/O mediator is a single hardware module — the only non-Church component — that:

1. Intercepts SAVE instructions targeting Golden Tokens with the F (Far) flag set, where the namespace entry identifies a physical device
2. Translates the Church-encoded output value into a physical bus transaction (UART write, GPIO toggle, LED update)
3. Intercepts LOAD instructions on device-class GTs and returns hardware status as Church-encoded values
4. Enforces capability permissions (R for read, W for write) on all device access

The I/O mediator is the architectural equivalent of mLoad for the physical world: a single trusted gate that cannot be bypassed.

### Processor Architecture

The pure Church processor comprises three hardware blocks:

1. **Lambda Reducer**: Executes LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM. Performs Church-encoded lambda reduction. Contains no arithmetic logic unit (ALU), no barrel shifter, no condition flag register, no branch prediction unit.
2. **Capability Validator**: Checks Golden Token permissions, verifies domain purity, validates version seals (FNV), performs the mLoad five-check validation sequence. This module is unchanged from the parent CTMM architecture.
3. **I/O Mediator**: Translates capability-secured SAVE/LOAD operations on device GTs (F=1) into physical bus transactions. The sole interface between pure lambda software and hardware peripherals.

### Security Properties Achieved by Construction

The architectural exclusion of Turing-domain instructions eliminates the following vulnerability classes **by construction** — not by mitigation, not by detection, but by the absence of the instructions needed to perform the attack:

1. **Buffer overflow**: Requires writing beyond allocated bounds via MOV/STORE to a computed address. No MOV or STORE to arbitrary addresses exists. SAVE writes only through a capability with verified permissions and bounds.
2. **Return-oriented programming (ROP)**: Requires chaining branch instructions to existing code gadgets. No branch instructions exist. CALL enters an abstraction scope determined by a Golden Token; LAMBDA applies a function referenced by a GT with X permission. Neither can jump to an arbitrary address.
3. **Code injection**: Requires writing executable code to a data region and branching to it. No instruction can write executable code (SAVE requires S permission, which is Church-domain and cannot coexist with X permission due to domain purity). No instruction can branch to an arbitrary address.
4. **Privilege escalation**: Requires manipulating kernel data structures or system call tables. No kernel exists (atomic abstraction architecture). No system call table exists. No instruction can forge

or modify a Golden Token.

5. **Use-after-free**: Requires accessing a memory region after its capability has been revoked. TPERM checks the GT before every operation; a revoked GT (version mismatch) causes an immediate FAULT.

6. **Confused deputy**: Requires tricking a privileged service into misusing its authority. Every operation verifies both the caller's capability permissions and the operation's permission requirements. TPERM makes the check explicit and unforgeable.

## REDUCTION TO PRACTICE

### Software Proof: HP-35 Scientific Calculator

The HP-35 implementation demonstrates computational completeness through 179 Church-domain instructions implementing 18 methods: digit entry, stack operations (ENTER, SWAP, CLR, CLX), four-function arithmetic (ADD, SUB, MUL, DIV), scientific functions (SIN, COS, LOG, EXP, SQRT, POW), sign change (CHS), and constant retrieval (PI via Abstract GT from Constants abstraction).

Verified by automated parsing: 179 instructions, 0 Turing-domain, 179 Church-domain. Opcode distribution: LOAD (60), RETURN (31), TPERM (30), CALL (27), LAMBDA (27), SAVE (4).

### Software Proof: SlideRule Arithmetic Engine

The SlideRule implementation demonstrates that the method-selector dispatch pattern works entirely in Church domain: 98 instructions implementing 9 methods (ADD, SUB, MUL, DIV, MOD, LOG, EXP, SQRT, POW) plus the access dispatcher.

Verified by automated parsing: 98 instructions, 0 Turing-domain, 98 Church-domain. Opcode distribution: LOAD (39), LAMBDA (27), TPERM (11), RETURN (11), CALL (10).

Notable: MOD is computed as SUB(a, MUL(b, DIV(a, b))) — composing three Church primitives without any Turing instruction. SQRT uses Y-combinator-driven linear search with Church LEQ comparison and SUCC/PRED stepping.

### Software Proof: Interactive Church Computer REPL

The Church Computer REPL is a Haskell implementation (~1,000 lines across 6 modules) that provides an interactive programming environment for the Pure Church Machine. It demonstrates that the architecture is not limited to pre-compiled programs but supports general-purpose interactive computation.

**Programming Model — Ada Lovelace Symbolic Mathematics**: The REPL accepts conventional mathematical notation — the same symbolic language Lovelace and Babbage exchanged daily — and translates it into capability-secured Church-domain operations. The programmer writes:

```
      let n = succ(3)                  -- n = 4
let n_plus_1 = succ(n)           -- n_plus_1 = 5
let product = n * n_plus_1       -- product = 20
let root = sqrt(product)         -- root = 4
```

The REPL translates each expression to its underlying Church-domain call and shows both:

```
      -- let product = n * n_plus_1  =>  Call(Lambda.MUL, 4, 5) --
 product = 20
```

Supported operators: `+`, `-`, `*`, `/`, `%`, `^` (mapped to SlideRule and Lambda abstractions). Supported functions: `sqrt()`, `log()`, `exp()`, `pow()`, `succ()`, `pred()`. Each line expresses one operation — matching Lovelace's style where each step produces a named result. This one-operation-per-line design is architectural: each expression maps to exactly one 7-step capability-checked pipeline traversal, making the security properties visible and auditable. The explicit `Call()` syntax remains available for direct abstraction access. The REPL also provides ANS (last result) for quick sequential computation, VARS (display all bindings), REGS (register state), and NS (namespace inspection).

**Bernoulli Demonstration**: A `.church` program file (Church/bernoulli.church) computes the sum of squares $1^2 + 2^2 + 3^2 + 4^2 = 30$ using Lovelace's step-by-step style: first via the closed-form formula $n \times (n+1) \times (2n+1)/6$, then via direct computation, verifying both paths produce 30. The program uses 17 named intermediate results across Lambda and SlideRule abstractions — all executed through pure Church-domain instructions with zero Turing instructions.

**Turing Rejection**: Any Turing-domain instruction (ADD, MOV, CMP, B, LDR, STR, PUSH, POP, etc.) entered at the REPL or encountered in a program file produces an immediate FAULT — the instructions do not exist in this architecture. This is not a filter or check; the instruction set literally does not include them.

**Error Handling**: Unknown variable references produce explicit error messages rather than silent failures, ensuring computational correctness. This mirrors the architecture's fail-safe security philosophy: every validation failure produces a visible fault, never a silent default.

### Hardware Proof: Synthesizable FPGA Implementation

The Sim-32 Amaranth HDL implementation (~3,150 lines, 18 modules) has been synthesized to Verilog (29,000 lines) and successfully placed on an iCE40 HX8K FPGA target: 1,982 LUTs (26% utilization), 1,132 flip-flops, 10 BRAMs (31%). The existing core implements both Church and Turing domains; modification to a Church-only core would reduce the design by removing the ALU, condition flags, branch logic, and barrel shifter — resulting in a smaller, simpler, and more formally verifiable design.

## PROPOSED CLAIMS

---

### Claim 17 — Pure Church Lambda Processor

A processor architecture comprising:

(a) a software-accessible instruction set consisting exclusively of lambda calculus reduction operations: LOAD (capability load from C-List with L permission), SAVE (capability-mediated write with S permission), CALL (abstraction scope entry with E permission), RETURN (scope exit), LAMBDA (in-scope function application with X permission), and TPERM (permission verification);

(b) wherein no arithmetic instruction (ADD, SUB, MUL, DIV), no logic instruction (AND, OR, XOR, shift, rotate), no comparison instruction (CMP, conditional test), no branch instruction (conditional or unconditional jump), no direct memory addressing instruction (load/store to computed address), and no register transfer instruction (MOV) is available to software;

(c) wherein all arithmetic computation is performed through Church-encoded lambda calculus reductions: Church numerals for natural numbers, Church booleans for conditional logic, Church pairs for structured data, and Y-combinator for recursive computation;

(d) wherein every instruction operates exclusively through Golden Tokens with hardware-verified permissions, and every failure routes to a single FAULT handler.

### Claim 18 — Security Enforcement Through Architectural Instruction Exclusion

The processor of Claim 17, wherein the architectural exclusion of Turing-domain instructions from the software instruction set eliminates, by construction rather than by mitigation:

(a) buffer overflow attacks, because no instruction can write to a computed arbitrary address;

(b) return-oriented programming attacks, because no branch instruction exists to chain code gadgets;

(c) code injection attacks, because no instruction can write executable code (domain purity prevents S and X permissions on the same GT) and no instruction can branch to an arbitrary address;

(d) privilege escalation, because no operating system, privilege rings, or superuser identity exists (atomic abstraction architecture) and no instruction can forge or modify a Golden Token;

(e) use-after-free exploits, because TPERM verifies capability validity (version seal) before every operation and revoked capabilities cause immediate FAULT.

### Claim 19 — Hardware I/O Mediator for Pure Lambda Processor

The processor of Claim 17, further comprising a hardware I/O mediator module that:

(a) is the sole hardware interface between pure lambda software and physical devices;

(b) intercepts SAVE instructions targeting Golden Tokens with the F (Far) flag set in the namespace entry, wherein the namespace entry identifies a physical device class;

(c) translates Church-encoded output values into physical bus transactions (register writes, GPIO operations, serial transmission);

(d) intercepts LOAD instructions on device-class Golden Tokens and returns hardware status as Church-encoded values;

(e) enforces Golden Token permissions (R for device read, W for device write) on all device access through the mLoad validation path;

(f) is architecturally equivalent to a single trusted gate for the physical world, and cannot be bypassed by any software instruction.

### Claim 20 — Church Numeral Method-Selector Dispatch

The processor of Claim 17, further comprising a method-selector dispatch mechanism wherein:

(a) a method selector value (DR1) is converted to a Church numeral by applying the Church successor function (GT_CHURCH_SUCC) DR1 times to Church zero (GT_FALSE), using the LAMBDA instruction;

(b) the resulting Church numeral is used to index into the abstraction's C-List to obtain the corresponding method GT;

(c) the method GT is verified via TPERM for Execute (X) permission and applied via LAMBDA;

(d) no branch instruction, no jump table, no computed goto, and no conditional logic instruction is used in the dispatch process;

thereby implementing polymorphic method dispatch entirely through lambda calculus without any Turing-domain instruction.

### Claim 21 — Church-Encoded Arithmetic Operations via Capability Tokens

The processor of Claim 17, wherein arithmetic operations including at least addition, subtraction, multiplication, division, modular arithmetic, exponentiation, logarithm, and square root are performed exclusively through:

(a) Church numeral encoding, wherein the natural number N is represented as the function that applies its first argument N times to its second argument;

(b) capability-mediated function application, wherein each arithmetic primitive (ADD, SUB, MUL, DIV, POW) is a Golden Token in the Lambda abstraction's C-List, loaded via LOAD with L permission and applied via LAMBDA with X permission;

(c) recursive operations (DIV, MOD, SQRT, LOG) use the Y-combinator Golden Token (GT_Y_COMBINATOR) for recursion, with Church LEQ for termination and Church SUCC/PRED for iteration;

(d) composite operations (MOD = SUB(a, MUL(b, DIV(a, b)))) are expressed by composing Church primitives through sequential LAMBDA applications, without any Turing-domain arithmetic instruction.

### Claim 22 — Three-Block Pure Church Processor Architecture

The processor of Claim 17, comprising exactly three hardware functional blocks:

(a) a Lambda Reducer that executes the six Church-domain instructions (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM), performs Church-encoded lambda reductions, and contains no arithmetic logic unit, no barrel shifter, no condition flag register, and no branch prediction unit;

(b) a Capability Validator that performs Golden Token permission verification, domain purity enforcement, version seal validation (FNV hash), and the mLoad five-check validation sequence for every namespace access;

(c) an I/O Mediator of Claim 19 that translates capability-secured lambda operations into physical bus transactions;

wherein blocks (a) and (b) implement pure Church lambda calculus computation secured by capabilities, and block (c) provides the sole interface to the physical world; and wherein the total processor comprises fewer functional units than a conventional processor (no ALU, no condition logic, no branch unit), resulting in a smaller silicon area, lower power consumption, and a design amenable to formal verification.

### Claim 23 — Interactive Pure Church Programming Model with Named Bindings

A method of programming the pure Church lambda processor of Claim 17, wherein all computations are executed by the pure Church instruction set of Claim 17 and maintain the security properties of Claim 18, comprising:

(a) an interactive execution environment (Read-Eval-Print Loop) wherein each user-supplied expression — whether expressed in conventional mathematical notation (infix operators, function calls) or explicit abstraction call syntax — is parsed, translated to capability-secured Church-domain operations, dispatched to the processor's six Church-domain instructions, executed through the complete capability-checked instruction pipeline (LOAD → TPERM → CALL → LOAD → TPERM → LAMBDA → RETURN), and the result displayed;

(b) named variable bindings, wherein the result of any Church-domain computation is stored with a programmer-assigned name and may be referenced as an argument in subsequent computations, following the step-by-step named-result programming style first described by Ada Lovelace in Note G (1843) for Babbage's Analytical Engine;

(c) program file execution, wherein a sequence of named-binding statements is loaded from a file and executed in order, with variable scope persisting across statements, enabling multi-step mathematical computations expressed entirely in Church-domain instructions;

(d) fail-safe error handling, wherein any reference to an undefined variable produces an explicit error

rather than a silent default, and any Turing-domain instruction produces an immediate FAULT — mirroring the architecture's hardware fail-safe behavior in software;

(e) wherein the programming model demonstrates that the pure Church lambda processor of Claim 17 supports general-purpose interactive programming, not merely fixed pre-compiled programs, while maintaining the security properties of Claim 18 for every computation.

## PRIOR ART DISTINCTION

| System | Lambda Reduction | Capability Security | Turing Excluded | Security via Exclusion |
|---|---|---|---|---|
| LISP Machines (MIT/Symbolics) | Yes | No | No | No |
| Cambridge CAP | No | Yes | No | No |
| IBM System/38 | No | Yes | No | No |
| CHERI (Cambridge) | No | Yes | No | No |
| Reduceron | Yes | No | No | No |
| GRIP | Yes | No | No | No |
| CTMM (parent application) | Yes | Yes | No | No |
| Pure Church Machine (this CIP) | Yes | Yes | Yes | Yes |

No prior system combines all four properties. The present invention is the first to use the architectural exclusion of Turing-domain instructions as a security enforcement mechanism within a capability-based architecture.

## FIGURES (Proposed)

### Figure 17: Pure Church Processor Block Diagram

Three-block architecture: Lambda Reducer (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM) connected to Capability Validator (mLoad, permission check, seal verify) connected to I/O Mediator (device GT translation). Single bus connecting to BRAM (C-Lists, namespace). No ALU block. No branch prediction block.

### Figure 18: Church Numeral Method Dispatch Flow

Flowchart: DR1 (method selector) → Enter Lambda scope (CALL) → Load SUCC and FALSE/ZERO → Apply SUCC DR1 times (LAMBDA) → Return to abstraction scope → Load indexed C-List slot → TPERM verify X → LAMBDA apply → RETURN result. No branch or jump instruction at any step.

### Figure 19: Vulnerability Elimination by Construction

Table mapping each vulnerability class (buffer overflow, ROP, code injection, privilege escalation, use-after-free, confused deputy) to the Turing instruction(s) required (MOV/STORE, branch, branch+STORE, MOV to kernel, LOAD after free, indirect CALL) and showing "Instruction does not exist" for each in the Pure Church architecture.

### Figure 20: HP-35 and SlideRule Opcode Distribution

Bar chart showing both implementations: only LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM used. Zero occurrences of ADD, SUB, MUL, MOV, CMP, B, or any other Turing-domain opcode.

### Figure 21: Ada Lovelace Programming Model — Bernoulli Sum-of-Squares

Annotated program listing showing the Church Computer executing a step-by-step computation following Lovelace's Note G style. The programmer writes symbolic mathematics (`let product = n * n_plus_1`) and the REPL shows the translation to capability-secured Church-domain operations (`=> Call(Lambda.MUL, 4, 5)`). 17 named variable bindings compute $1^2 + 2^2 + 3^2 + 4^2 = 30$ two ways (closed-form formula and direct summation). Annotations highlight: (a) conventional mathematical notation is the programming interface, (b) every operation translates to pure Church-domain lambda calculus underneath, (c) variables reference previous results by name, (d) Turing instructions (ADD, MUL, POW) do not exist — the symbolic operators +, *, ^ map to Church-encoded abstractions accessed via Golden Tokens.